

Beginner Guide to Generate Code Coverage Report in Spring Boot with IntelliJ IDEA Built-in Coverage Tool

1. Introduction

In modern software development, it is important to test the application properly.

But only running test cases is not enough. We also need to know **how much code is actually tested**.

For this purpose, **code coverage tools** are used.

2. What is Code Coverage

Code coverage means:

How much part of your source code is executed when your test cases run.

For example:

- If your project has 100 lines of code
- And your test cases execute 70 lines
- Then your code coverage is **70%**

In a Spring Boot project, one of the most convenient tools for checking code coverage is **IntelliJ IDEA's built-in Coverage Tool** — no external plugin or configuration required.

3. What is IntelliJ Built-in Code Coverage Tool

IntelliJ IDEA comes with a **built-in code coverage runner** that works out of the box — no Maven plugin or external library needed.

It is integrated directly inside the IDE and supports:

- JUnit and TestNG test frameworks
- Line coverage
- Branch coverage
- Method and class coverage

It generates a coverage report that shows:

- **Covered lines** — lines executed during tests
- **Missed lines** — lines not executed

- **Covered methods** — methods called during tests
- **Branch coverage** — if-else conditions tested

4. Requirements

Before generating the coverage report, make sure you have:

- Java installed
- IntelliJ IDEA installed (Community or Ultimate edition)
- Spring Boot project opened in IntelliJ
- At least one test case written in the project
- Main source code inside `src/main/java`
- Test cases inside `src/test/java`

5. No Plugin Required

Unlike JaCoCo, the IntelliJ built-in coverage tool **does not require any changes to pom.xml**. You do **not** need to add any plugin or dependency. The tool is already available inside IntelliJ IDEA.

However, make sure your test framework dependency is present in **pom.xml**:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

6. Make Sure Test Cases Are Present

IntelliJ coverage report is generated based on test execution.

That means:

- If no test case is present, coverage will be **0%** or very low.
- So, your project should have at least one test case.

7. Steps to Run with Code Coverage in IntelliJ IDEA

Follow these steps to generate the coverage report:

Step 1: Open Your Project in IntelliJ IDEA

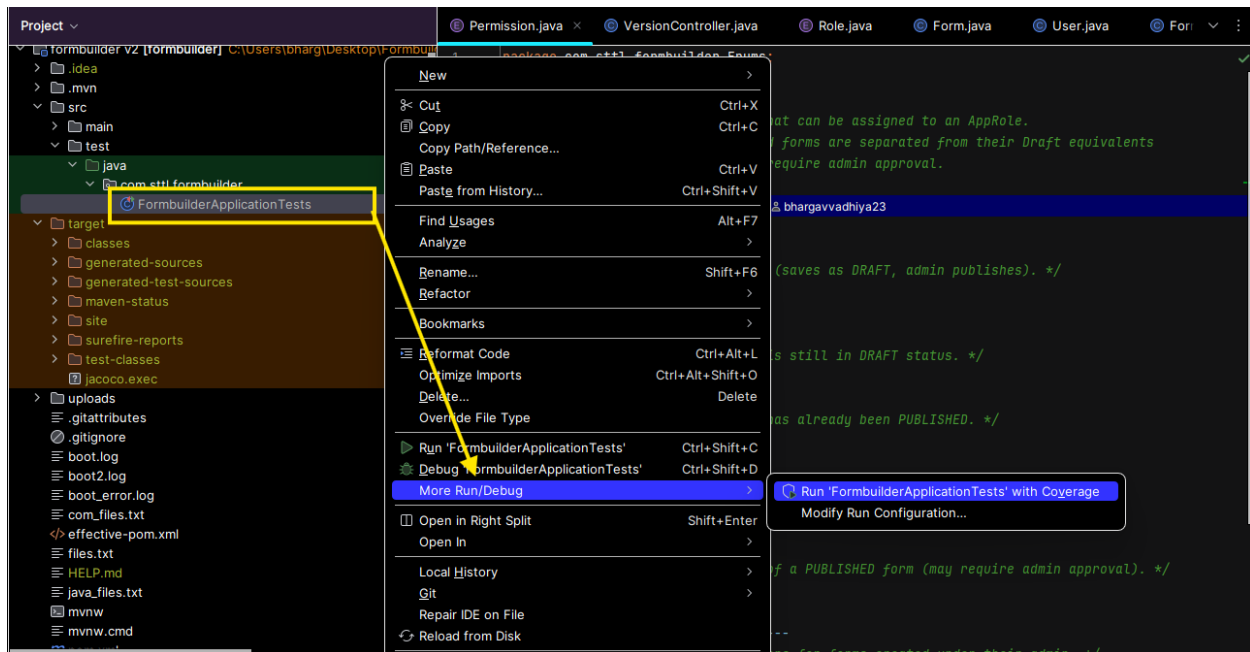
Open your Spring Boot project in IntelliJ IDEA.

Step 2: Right-click and Run with Coverage

Right-click on the Main class or the **src/main/java** folder.

Select **"More Run/Debug" → "Run '{your_App_Name}' with Coverage"**.

OR — open the **Run** menu from the top bar and select **"Run '...' with Coverage"**.



Step 3: Manually use of Application to generate Report

When an application is launched using **Run with Coverage** in IntelliJ IDEA, the IDE tracks the code that is executed during runtime. This means test cases are not mandatory for collecting coverage data, as manual usage of the application can also generate coverage. Only the code paths that are actually executed are marked as covered.

Example:

1. Start the Spring Boot application using **Run with Coverage**.
2. Use the application manually through a browser, frontend, or Postman.
3. The controllers, services, and methods that are invoked will be shown as covered in the coverage report.

Important Note:

However, coverage is recorded only when the application is started with **Run with Coverage**. If it is run normally, no coverage data is collected. Since manual usage usually covers only selected flows, some conditions, exception paths, and edge cases may remain uncovered.

Step 4: View Coverage Results

After the test run completes, IntelliJ will automatically:

- Highlight covered lines in **green** in the editor
- Show missed lines in **red**
- Open the **Coverage** tool window (usually on the right or bottom panel)

The Coverage tool window shows a tree of packages and classes with coverage percentages.

Element	Class, %	Method, %	Line, %	Branch, %
com.sttl.formbuilder	70% (40/57)	13% (58/427)	8% (195/2349)	0% (2/2176)
FormbuilderApplication	100% (1/1)	100% (1/1)	100% (1/1)	100% (0/0)
util	0% (0/2)	0% (0/10)	0% (0/101)	0% (0/91)
service	70% (12/17)	14% (16/112)	4% (45/1024)	0% (1/676)
repository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
model	0% (0/1)	0% (0/25)	0% (0/37)	0% (0/6)
exception	100% (1/1)	20% (1/5)	5% (2/34)	0% (0/16)
Enums	100% (2/2)	100% (4/4)	100% (10/10)	100% (0/0)
entity	90% (9/10)	9% (11/117)	28% (42/149)	0% (0/740)
dto	0% (0/4)	0% (0/35)	0% (0/41)	0% (0/242)
controller	71% (10/14)	9% (10/101)	4% (36/886)	0% (0/403)
config	100% (5/5)	88% (15/17)	89% (59/66)	50% (1/2)

8. How to Generate an HTML Report

IntelliJ also lets you export the coverage report as an HTML file.

Follow these steps:

- After running with coverage, click on the **Coverage** tool window
- Click the **Export Report** icon (or go to **Analyze > Export Coverage Report**)
- Choose the output directory and format (HTML)
- Click **Save**

The report will be saved as an **index.html** file in the selected folder.

Open this file in a browser to view the full coverage report.

9. What Sections Are Visible Inside the Report

When you open the IntelliJ coverage report, you will see these columns:

- **Class, %** — how many classes are covered
- **Method, %** — how many methods are tested
- **Branch, %** — how many conditions like if-else are tested
- **Line, %** — how many source code lines are covered

Current scope: all classes

Overall Coverage Summary

Package	Class, %	Method, %	Branch, %	Line, %
all classes	86.1% (87/101)	77.9% (282/362)	15% (266/1772)	74.2% (1178/1588)

Coverage Breakdown

Package	Class, %	Method, %	Branch, %	Line, %
com.sttl.formbuilder2	100% (1/1)	100% (2/2)		100% (2/2)
com.sttl.formbuilder2.config	100% (1/1)	100% (2/2)		100% (6/6)
com.sttl.formbuilder2.controller	100% (13/13)	74.7% (59/79)	47.4% (36/76)	73.3% (255/348)
com.sttl.formbuilder2.dto.internal	0% (0/4)	0% (0/5)	0% (0/100)	0% (0/5)
com.sttl.formbuilder2.dto.request	100% (7/7)	100% (7/7)	0% (0/318)	100% (7/7)
com.sttl.formbuilder2.dto.response	95.8% (23/24)	90.5% (38/42)	1.5% (11/740)	90.5% (38/42)
com.sttl.formbuilder2.exception	100% (1/1)	100% (4/4)		100% (8/8)
com.sttl.formbuilder2.model.entity	82.6% (19/23)	77.5% (55/71)	15.8% (6/38)	78.5% (73/93)
com.sttl.formbuilder2.model.enums	40% (2/5)	28.6% (2/7)		63% (29/46)
com.sttl.formbuilder2.security	100% (3/3)	87.5% (14/16)	5% (1/20)	77.8% (49/63)
com.sttl.formbuilder2.service	89.5% (17/19)	78% (99/127)	44.2% (212/480)	73.5% (711/968)

generated on 2026-03-18 11:26

10. Coverage Report — formbuilder2 Project

The following table shows the coverage report generated by IntelliJ built-in tool for the **formbuilder2** Spring Boot project (generated on 2026-03-18):

Package	Class, %	Method, %	Branch, %	Line, %
com.sttl.formbuilder2	100% (1/1)	100% (2/2)	—	100% (2/2)
com.sttl.formbuilder2.config	100% (1/1)	100% (2/2)	—	100% (6/6)
com.sttl.formbuilder2.controller	100% (13/13)	74.7% (59/79)	47.4% (36/76)	73.3% (255/348)
com.sttl.formbuilder2.dto.internal	0% (0/4)	0% (0/5)	0% (0/100)	0% (0/5)
com.sttl.formbuilder2.dto.request	100% (7/7)	100% (7/7)	0% (0/318)	100% (7/7)
com.sttl.formbuilder2.dto.response	95.8% (23/24)	90.5% (38/42)	1.5% (11/740)	90.5% (38/42)
com.sttl.formbuilder2.exception	100% (1/1)	100% (4/4)	—	100% (8/8)
com.sttl.formbuilder2.model.entity	82.6% (19/23)	77.5% (55/71)	15.8% (6/38)	78.5% (73/93)
com.sttl.formbuilder2.model.enums	40% (2/5)	28.6% (2/7)	—	63% (29/46)

Package	Class, %	Method, %	Branch, %	Line, %
com.sttl.formbuilder2.security	100% (3/3)	87.5% (14/16)	5% (1/20)	77.8% (49/63)
com.sttl.formbuilder2.service	89.5% (17/19)	78% (99/127)	44.2% (212/480)	73.5% (711/968)
ALL CLASSES (Overall)	86.1% (87/101)	77.9% (282/362)	15% (266/1772)	74.2% (1178/1588)

11. Conclusion

IntelliJ IDEA's built-in coverage tool is a very convenient way to measure test coverage in Spring Boot applications.

Without adding any external plugin or modifying **pom.xml**, you can run your test cases with coverage directly inside IntelliJ and view a detailed report.

This report helps developers identify:

- which code is tested
- which code is not tested
- where additional test cases are required

Therefore, the IntelliJ built-in coverage tool improves testing quality and helps in building more reliable Spring Boot applications.